Everything NOT Covered — The Complete Deep Dive

Let me cover every concept that was either skipped, glossed over, or only surface-level mentioned in my previous answer.

---

1. WHY Contiguous Memory Allocation? — The Deep "Why"

I told you arrays are contiguous. But I never deeply explained **why Java chose this design** and what the alternative looks like.

The Alternative — Non-Contiguous Storage (Linked List style)

Imagine storing numbers scattered anywhere in memory:

```
Address 100: value = 10, "next element is at address 847"
Address 847: value = 20, "next element is at address 3291"
Address 3291: value = 30, "next element is at address 12"
Address 12:   value = 40, "next element is at address 2000"
```

To find element at index 3, you MUST:

- Start at 100 (index 0)
- Follow pointer to 847 (index 1)
- Follow pointer to 3291 (index 2)
- Follow pointer to 12 (index 3) ← FINALLY found it

This is **O(n)** — to find index 3, you made 4 jumps. To find index 1000, you make 1001 jumps. As array grows, access time grows proportionally.

Why Contiguous Fixes This Completely

With contiguous memory:

- You don't need to store "where is the next element"
- You don't need to follow any pointers

- Pure arithmetic gets you anywhere instantly

The formula `base + (size × index)` works **only because** elements are sitting side by side with no gaps. Remove the contiguity, the formula becomes impossible to construct.

The Real Cost of Contiguous Allocation — What I Didn't Tell You

Here is something important that was never mentioned: **contiguous allocation has a hidden cost at creation time.**

When you write `new int[1000000]` (one million integers = 4MB), the JVM must find **4 million consecutive bytes free** in the heap. Not just 4 million bytes total — they must be back to back.

If your heap looks like this (F = free, U = used):

UUUUFFUUUUFFFUUFFFFFFFFFFFUUUUUFFFFFFFFFFF...

Even if total free space is 4MB, if no single contiguous block is 4MB, your allocation **fails** with an `OutOfMemoryError`. This is a real-world problem called **heap fragmentation** and it's one reason garbage collection exists — to compact memory and create larger contiguous free blocks.

This is the fundamental trade-off of arrays:

- **Fast to access** (O(1) always)
- **Expensive to create** (needs contiguous block)
- **Fixed size forever** (can't grow without creating a brand new array)

---

2. The `new` Keyword — What It ACTUALLY Does Step by Step

I said "things created with `new` go to heap" but never explained what `new` actually triggers internally.

When you write `new int[5]`, this is the exact sequence of events:

Step 1: Class Loading Check

The JVM checks if the type (`int[]` in this case) is already loaded. For primitive arrays this is trivial, but for object arrays this involves loading the class definition.

Step 2: Memory Calculation

JVM calculates: How many bytes do I need?

Total bytes = (number of elements × size of each element) + header overhead

Wait — **header overhead**? Yes. Every object in the heap (and arrays ARE objects in Java) has an **object header**. This was never mentioned before.

The object header contains:

- **Mark Word** (~8 bytes): stores hash code, garbage collection information, lock status
- **Class Pointer** (~4 or 8 bytes): points to the class definition (what type is this array?)
- **Array Length** (~4 bytes): this is where `arr.length` comes from — it's stored RIGHT HERE in the header

So `new int[5]` actually allocates:

Header (~16 bytes) + (5 × 4 bytes) = ~36 bytes total

Not just 20 bytes. The header is invisible to you but very real.

Step 3: Heap Allocation

JVM finds a contiguous free block of the required size in heap memory and claims it.

Step 4: Zero Initialization

This is critical and often forgotten. Java **guarantees** that every newly created array is initialized to default values:

- `int[]` → all zeros
- `boolean[]` → all false
- `double[]` → all 0.0
- `char[]` → all '\u0000' (null character)
- `String[]` → all null
- Any object array → all null

This zero-initialization is why Java is memory safe. In C/C++, `new int[5]` gives you whatever garbage was previously at that memory location. Java always wipes it clean first.

Step 5: Reference Returned

The starting address of the newly allocated block is returned and stored in your reference variable (`arr`).

---

3. Stack Memory — The Deep Explanation

I mentioned stack memory stores primitives and references, but never explained how it actually works.

The Call Stack Structure

Every time a method is called in Java, a **stack frame** is pushed onto the stack. When the method finishes, the frame is **popped** (removed) and that memory is instantly freed.

```
public static void main(String[] args) {
    int x = 4;              // lives in main's stack frame
    int[] arr = new int[5]; // reference lives in main's stack frame
                            // actual array lives in heap
    doSomething(arr);
}

public static void doSomething(int[] a) {
    int y = 10;   // lives in doSomething's stack frame
    // when this method ends, y and 'a' reference are destroyed
    // but the heap array continues to exist
}
```

The stack looks like:

```
┌──────────────────────────────────────┐ ← top of stack
│  doSomething's frame                 │
│  a = 100 (reference)                 │
│  y = 10                              │
├──────────────────────────────────────┤
│  main's frame                        │
│  x = 4                               │
│  arr = 100 (reference)               │
│  args = some_address                 │
└──────────────────────────────────────┘ ← bottom of stack
```

When `doSomething` returns:

```
┌──────────────────────────────────────┐ ← top of stack
│  main's frame                        │
│  x = 4                               │
│  arr = 100 (reference)               │
│  args = some_address                 │
└──────────────────────────────────────┘
```

The frame for `doSomething` is gone. Instantly. No garbage collection needed. This is why stack allocation/deallocation is so fast.

Why Stack Has Size Limits

Stack memory is small (typically 512KB to 1MB by default in Java). This is why infinite recursion causes a `StackOverflowError` — you keep pushing frames until the stack is full.

```
public static void infinite() {
    infinite(); // keeps adding frames
}
// Eventually: java.lang.StackOverflowError
```

Heap memory is much larger (can be gigabytes, configured with `-Xmx` JVM flag).

The Critical Point About References Dying

When `doSomething` finishes, the reference `a` is destroyed (popped off stack). But the actual array in the heap is still there. Nobody told it to leave. This is where the garbage collector comes in — it periodically checks: "Is there any reference variable anywhere pointing to this heap object?" If not, it's garbage and can be collected.

This is why you can't have memory leaks in Java (well, you can, but it's harder) — the GC handles cleanup automatically.

---

4. Reference Variables — The Complete Picture

I said reference variables "store an address" and "point to" something. Let me make this absolutely precise.

A Reference Variable is Just a Number

That's literally all it is. A reference variable stores an integer — the memory address of the heap object it refers to.

```
int[] arr = new int[5];
```

If the array lands at heap address 100, then `arr` contains the number `100`. That's it. The "magic" of references is just the JVM knowing: "when I see this variable, treat the number inside it as a memory address, not as a value."

What Happens With Assignment

```
int[] arr1 = new int[5];  // arr1 = 100 (heap address)
int[] arr2 = arr1;        // arr2 = 100 (SAME heap address)

Stack:                    Heap:
arr1 = 100 ──┐
             ├──→ [0][0][0][0][0]  (at address 100)
arr2 = 100 ──┘
```

Both `arr1` and `arr2` point to the **exact same array**. Now:

```
arr2[0] = 999;
System.out.println(arr1[0]); // prints 999 !!!
```

This shocks beginners. Modifying through `arr2` affects `arr1` because they're the same array. You copied the reference (the address number), not the array contents.

This Is Why Passing Arrays to Methods Works This Way

```
public static void modify(int[] a) {
    a[0] = 999; // modifies the ORIGINAL array
}

int[] arr = new int[5];
modify(arr); // passes the address, not a copy
System.out.println(arr[0]); // 999
```

Java is "pass by value" but the value being passed for non-primitives is the **address**. So you're passing a copy of the address, which still points to the original heap object.

---

5. How `arr.length` Works — The Real Mechanism

Every time you write `arr.length`, where does that number come from?

I mentioned it's stored in the object header. Here is the precise detail:

When `new int[5]` executes, the JVM allocates the memory block and writes `5` into the length field of the header. This number is **immutable** — it can never change for the lifetime of that array object.

Memory at address 100:

```
┌──────────────────────┐
│ Mark Word (8 bytes)  │ ← GC info, hash, lock state
│ Class Ptr (4 bytes)  │ ← "this is an int array"
│ Length: 5 (4 bytes)  │ ← arr.length reads THIS
├──────────────────────┤
│ element[0] (4 bytes) │ ← actual data starts here
│ element[1] (4 bytes) │
│ element[2] (4 bytes) │
│ element[3] (4 bytes) │
│ element[4] (4 bytes) │
└──────────────────────┘
```

`arr.length` is not a method call — there are no parentheses after it. It's a **field access** that reads directly from the header. This is O(1) and extremely fast.

This is also why arrays in Java can never be resized. The length is written once at creation, it's stored in the fixed-size header, and it's immutable. To "resize" you must create a brand new array and copy elements over (which is exactly what `ArrayList` does internally — this was never explained but is crucial to understanding why `ArrayList` exists).

---

6. The ArrayIndexOutOfBoundsException — Internal Implementation Details

I showed the conceptual check but not how it's actually implemented inside the JVM.

The JVM has two strategies, and it uses BOTH:

Strategy 1: Software Bounds Check (what we discussed)

Before every array access, the JVM inserts a bounds check in the bytecode. But doing this for every single access adds overhead.

Strategy 2: Hardware-Level Memory Protection (what I didn't mention)

Modern JVMs (especially with JIT compilation) are smarter. The **Just-In-Time (JIT) compiler** can analyze your code:

```
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]); // bounds check here
}
```

The JIT compiler sees: "i starts at 0, goes up to arr.length - 1, and arr.length never changes in this loop." So it can **eliminate the bounds check entirely** inside the loop — it proves mathematically that the index can never be out of bounds. This optimization is called **bounds check elimination**.

This is one reason Java arrays can be nearly as fast as C arrays in tight loops, even with the theoretical overhead of bounds checking.

---

7. Memory Addresses — How They Actually Work

I kept saying "address 100, address 104" but never explained what memory addresses actually are.

Virtual Memory — The Address Isn't Real

The addresses (100, 104, etc.) are **virtual addresses**, not physical RAM addresses. Every process running on your computer gets its own **virtual address space** — an imaginary, private memory space.

Your Java program thinks it has memory from address 0 to 2^64

The OS maps these virtual addresses to actual physical RAM
Two programs can both think they own address 100 — that's fine
The OS and CPU's Memory Management Unit (MMU) translates virtual → physical

This means:

- Array address 100 in your JVM is not physical RAM location 100
- The OS handles the translation transparently
- This is why multiple programs can run simultaneously without interfering

Why Addresses Are Always Multiples of 4 (or 8)

You might have noticed in the examples, addresses jump by 4 (100, 104, 108...) for int arrays. This isn't coincidence — it's **memory alignment**.

CPUs are most efficient when reading data that starts at addresses divisible by the data size. Reading a 4-byte int from address 103 (not divisible by 4) requires TWO separate memory reads and a combine operation on some hardware. Reading from address 104 (divisible by 4) requires ONE memory read.

Java's JVM ensures primitive data is always aligned to its natural size:

- `int` (4 bytes) → always at addresses divisible by 4
- `long` (8 bytes) → always at addresses divisible by 8
- `char` (2 bytes) → always at addresses divisible by 2

This alignment is built into the array memory layout automatically.

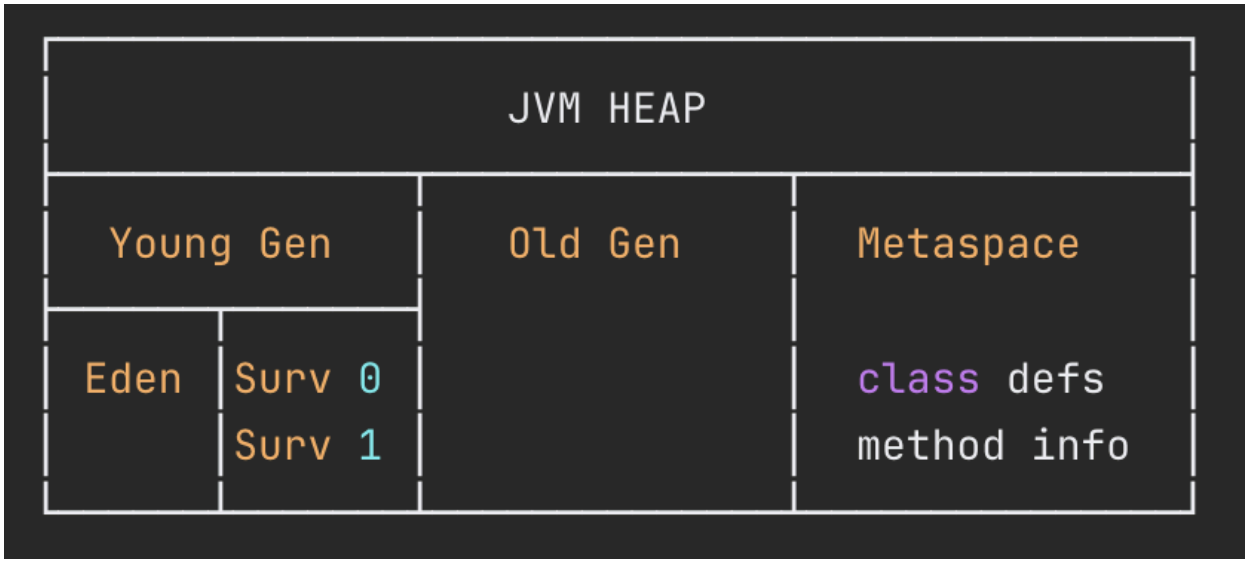
---

8. The Heap — Much More Detail

I said "heap is a large warehouse" but that's oversimplified.

The Heap is Divided Into Regions

Modern JVM heaps (using the G1 or ZGC garbage collector) divide the heap into regions:



Young Generation (Eden + Survivor spaces):

- New objects start here
- Your `new int[5]` lands in Eden space first
- Most objects die young (a local variable array used briefly)
- Garbage collected frequently (Minor GC) — fast because most of Eden is garbage

Old Generation:

- Objects that survive multiple Minor GCs get promoted here
- Your array survives if it's still referenced after several GC cycles
- Garbage collected infrequently (Major GC) — slow because less garbage

Why does this matter for arrays? A large array (`new int[10000000]`) might skip Eden entirely and go directly to Old Gen because it's too large for Eden space. This is called **direct allocation to Old Generation** or going into the **Humongous region** in G1 GC.

---

9. What `null` Actually Is for Arrays

This was never mentioned. What happens here:

```
int[] arr = null;
System.out.println(arr[0]); // What happens?
```

`null` means the reference variable holds the value `0` (or some special sentinel value meaning "no address"). When you try to access `arr[0]`, the JVM tries to go to address 0 to find the array header, but address 0 is never a valid heap address (it's reserved precisely for this purpose). The JVM catches this and throws a `NullPointerException`.

```
int[] arr; // declared but not initialized
// arr holds 'null' by default (for instance variables)
// For local variables, Java forces you to initialize before use
```

The difference:

```
// Instance variable - automatically null
class MyClass {
    int[] arr; // null by default, no NPE yet
}

// Local variable - must initialize
public void method() {
    int[] arr; // compiler error if you try to use arr without assigning
    System.out.println(arr[0]); // "variable arr might not have been initialized"
}
```

---

10. Array Copying — The Deep Mechanics

When people want to "resize" an array, they copy. Three ways, with very different internals:

Method 1: Manual Loop

```
int[] original = {1, 2, 3, 4, 5};
int[] copy = new int[10];
for (int i = 0; i < original.length; i++) {
    copy[i] = original[i];
}
```

This is pure Java — slow because it goes through the JVM for every element.

Method 2: `System.arraycopy()`

```
System.arraycopy(original, 0, copy, 0, original.length);
```

This is a **native method** — implemented in C inside the JVM. It uses the CPU's memory copy instructions directly (`memcpy`). It bypasses all Java-level overhead and copies raw bytes at hardware speed. For large arrays, this can be 10–100× faster than a manual loop.

Method 3: `Arrays.copyOf()`

```
int[] copy = Arrays.copyOf(original, 10);
```

Internally, this calls `System.arraycopy`. It's just a convenience wrapper.

Shallow vs Deep Copy — Critical Distinction

For primitive arrays, copying is always a **deep copy** — you get entirely independent data:

```
int[] a = {1, 2, 3};
int[] b = Arrays.copyOf(a, 3);
b[0] = 999;
System.out.println(a[0]); // still 1, completely independent
```

For object/reference arrays, copying is a **shallow copy** — you copy the references, not the objects themselves:

```
int[][] a = {{1, 2}, {3, 4}};
int[][] b = Arrays.copyOf(a, 2);
b[0][0] = 999;
System.out.println(a[0][0]); // 999 !!!
// b[0] and a[0] point to the SAME inner array
```

This is because copying an array of references copies the address numbers — both the original and the copy now have references pointing to the same inner arrays in the heap.

---

11. How Jagged Arrays Work in Depth

```
int[][] jagged = new int[3][];
jagged[0] = new int[2];
jagged[1] = new int[5];
jagged[2] = new int[1];
```

Memory layout:

Stack:
jagged = 500

Heap at 500 (outer array, 3 references × 4 bytes):

```
┌──────────┬──────────┬──────────┐
│  600     │  700     │  800     │
└──────────┴──────────┴──────────┘
```

Heap at 600: [0][0] ← 2 elements
Heap at 700: [0][0][0][0][0] ← 5 elements
Heap at 800: [0] ← 1 element

The outer array doesn't know or care that the inner arrays have different lengths. It just stores references. Each inner array stores its OWN length in its own header.

This is impossible in languages with true 2D arrays (like C's `int[3][4]`) because the formula for access would break — it assumes every row has the same length.

---

12. Multi-dimensional Array Creation — What Happens Internally

int[][] arr = new int[3][4];

Java executes this as if you wrote:

```
int[][] arr = new int[3][];      // Step 1: create outer array
arr[0] = new int[4];             // Step 2: create and assign row 0
arr[1] = new int[4];             // Step 3: create and assign row 1
arr[2] = new int[4];             // Step 4: create and assign row 2
```

Four separate heap allocations happen. Four separate zero-initialization passes. The outer array and three inner arrays are completely separate heap objects.

This is very different from C where `int arr[3][4]` is ONE contiguous block of 48 bytes.

---

13. Type Safety in Arrays — What Happens at Runtime

Object[] arr = new String[3];

```
arr[0] = "Hello";    // fine
arr[0] = 42;         // throws ArrayStoreException at RUNTIME
```

Wait — why does `arr[0] = 42` fail? `arr` is declared as `Object[]` and integers are objects (autoboxed to `Integer`). The answer: Java arrays carry their **actual type** in the object header at runtime. Even though the reference `arr` is typed as `Object[]`, the actual heap object is a `String[]`, and the JVM enforces that only Strings can be stored in it. This check happens at runtime and throws `ArrayStoreException`.

This is called **array covariance** and is actually considered a flaw in Java's type system — it allows type-unsafe code to compile but fail at runtime.

---

14. The JIT Compiler and Array Access Optimization

The JVM doesn't always interpret bytecode — the **JIT (Just-In-Time) compiler** watches which code runs frequently ("hot code") and compiles it to native machine code.

For arrays, the JIT does several optimizations that were never mentioned:

Loop Unrolling

```
for (int i = 0; i < 4; i++) {
    arr[i] = i * 2;
}
```

JIT might convert this to:

```
arr[0] = 0;  // no loop overhead, no bounds check
arr[1] = 2;
arr[2] = 4;
arr[3] = 6;
```

Vectorization (SIMD)

For numeric array operations, the JIT can use CPU SIMD (Single Instruction, Multiple Data) instructions — one CPU instruction that processes multiple array elements simultaneously.

```
// You write:
for (int i = 0; i < arr.length; i++) {
    arr[i] = arr[i] * 2;
```

```
}

// JIT might use AVX2 instructions to multiply 8 integers simultaneously
// Effectively 8× faster than one-at-a-time processing
```

This is only possible because arrays are contiguous — SIMD instructions require contiguous memory to operate on multiple elements at once.

---

15. String Interning — The Hidden String Pool

This was mentioned briefly but never explained. There is a special region of memory for strings.

```
String a = "Hello";
String b = "Hello";
System.out.println(a == b); // true !!! same object !!!
```

When you write a string literal like `"Hello"`, Java checks a **String Pool** (also called intern pool, lives in heap since Java 7). If `"Hello"` already exists in the pool, Java reuses the same object instead of creating a new one.

```
String Pool in Heap:
┌──────────────────┐
│  "Hello"         │ ← one object shared by a and b
│  "World"         │
│  "Java"          │
└──────────────────┘

Stack:
a = address of "Hello" in pool
b = address of "Hello" in pool  ← SAME address as a
```

But:

```
String c = new String("Hello");
System.out.println(a == c); // false !!! different objects !!!
```

Using `new String()` bypasses the pool and forces creation of a new String object in the regular heap. `a` points to the pooled string, `c` points to a separate new string.

This is why you should ALWAYS compare strings with `.equals()` not `==`:

- `==` compares addresses (are these the same object?)
- `.equals()` compares content (do these contain the same characters?)

---

16. The Full Picture of Primitive vs Reference in Method Arguments

This distinction is so important that it deserves its own complete section.

```
// Primitive - original is NEVER affected
public static void changeInt(int x) {
    x = 999; // only changes the local copy
}

int n = 5;
changeInt(n);
System.out.println(n); // still 5
```

What happened in memory:

```
Before call:
Stack: n = 5

During call:
Stack: n = 5
       x = 5  ← a copy of the VALUE 5
x = 999 changes only x, n untouched

After call:
Stack: n = 5  ← unchanged
(x frame popped and gone)
```

```
// Reference - array CAN be modified through the reference
public static void changeArray(int[] arr) {
    arr[0] = 999;
}

int[] myArr = {1, 2, 3};
changeArray(myArr);
```

```
System.out.println(myArr[0]); // 999 !!!
```

What happened in memory:

```
Before call:
Stack: myArr = 100  (pointing to heap)
Heap 100: [1][2][3]

During call:
Stack: myArr = 100
      arr = 100  ← a copy of the ADDRESS 100 (not a copy of the array)

arr[0] = 999 follows the address 100 and modifies the heap object

After call:
Stack: myArr = 100
Heap 100: [999][2][3]  ← permanently modified
```

But you **cannot** make `myArr` itself point somewhere else from inside a method:

```
public static void redirect(int[] arr) {
   arr = new int[]{10, 20, 30}; // arr now points to new array
   // but myArr in the caller still points to the original!
}

int[] myArr = {1, 2, 3};
redirect(myArr);
System.out.println(myArr[0]); // still 1
```

Because `arr = new int[]{...}` changes the local copy of the address inside the method's stack frame, not the original `myArr` reference in the caller's stack frame.

---

17. Cache Lines — The Precise Technical Detail

I mentioned CPU caching but didn't give the precise mechanics.

A **cache line** is 64 bytes on virtually all modern CPUs (x86, ARM, etc.). Whenever you access any memory location, the CPU fetches the entire 64-byte block containing that address.

For `int[]`:

- Each int = 4 bytes
- 64 bytes / 4 bytes = **16 integers per cache line**
- Accessing `arr[0]` brings `arr[0]` through `arr[15]` into cache simultaneously

For `long[]`:

- Each long = 8 bytes
- 64 bytes / 8 bytes = **8 longs per cache line**

This is why iterating through an array forwards is faster than jumping around randomly — forward iteration continuously hits cached data (cache hits), while random access constantly needs fresh cache loads (cache misses).

Concrete Performance Difference

```
// Fast - sequential access, cache-friendly
for (int i = 0; i < arr.length; i++) {
    sum += arr[i]; // cache hit after first element of each cache line
}

// Slow - random access, cache-unfriendly
for (int i = 0; i < arr.length; i++) {
    sum += arr[random_index[i]]; // constant cache misses
}
```

The random version can be 10–50× slower on large arrays, purely due to cache behavior. The algorithmic complexity (O(n)) is identical. This is why understanding memory layout matters for real performance.

2D Array Cache Behavior — Row-Major vs Column-Major

```
int[][] arr = new int[1000][1000];

// Fast - row by row (cache friendly)
for (int i = 0; i < 1000; i++)
    for (int j = 0; j < 1000; j++)
        sum += arr[i][j]; // sequential in memory

// Slow - column by column (cache hostile)
for (int j = 0; j < 1000; j++)
    for (int i = 0; i < 1000; i++)
        sum += arr[i][j]; // jumps 1000 elements between accesses
```

Java's 2D array stores each row as a separate array. Within a row, elements are contiguous. Between rows, there are gaps (different heap allocations). So iterating row by row is cache-friendly (sequential within each row), while iterating column by column jumps to a completely different heap location for every single access.

This can make a 10× performance difference on large matrices — and it's entirely explained by the memory layout we discussed.

---

18. `ArrayList` — Why It Exists and How It Relates

Since arrays have fixed sizes, Java provides `ArrayList` which "grows." How?

```
ArrayList<Integer> list = new ArrayList<>();
list.add(1);
list.add(2);
// ... add 1000 elements
```

Internally, `ArrayList` contains a regular `int[]` (actually `Object[]`). When you add elements:

1. If internal array has space → insert element, increment size counter
2. If internal array is FULL → create new array (typically 1.5× the current size), `System.arraycopy` all elements over, discard old array reference (GC will clean it up), continue

```
Initial: Object[] of size 10, 0 elements used
Add 10 elements: array full
Grow: new Object[] of size 15, copy 10 elements, now 11 used
Add 4 more: 15 used
Grow: new Object[] of size 22, copy 15, now 16 used
```

The amortized cost of `add()` is O(1) — most adds are instant (array has space), occasional grows are O(n) but happen so rarely they average out to constant time.

The point: `ArrayList` is entirely built on top of arrays. Understanding arrays deeply means you understand ArrayList's performance characteristics too.

Putting Every Concept Together — Complete Memory Picture

```
public static void main(String[] args) {
    int x = 4;
    int[] arr = new int[3];
    arr[0] = 10;
    arr[1] = 20;
    arr[2] = 30;

    String[] names = new String[2];
    names[0] = "Alice";
    names[1] = "Bob";

    int[][] matrix = new int[2][3];
}
```

Complete memory picture:

```
┌──────────────────────────────────────────────────────────┐
│                        STACK (main frame)                │
├──────────────────────────────────────────────────────────┤
│ x = 4           (primitive, value stored directly)       │
│ arr = 1000      (reference, address of heap array)       │
│ names = 2000    (reference, address of heap array)       │
│ matrix = 3000   (reference, address of heap array)       │
└──────────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────────┐
│                              HEAP                            │
├──────────────────────────────────────────────────────────────┤
│ addr 1000: [header|length=3] [10] [20] [30]                  │
│            (arr's array)                                     │
│                                                              │
│ addr 2000: [header|length=2] [4000] [5000]                   │
│            (names' array - stores references)                │
│                                                              │
│ addr 4000: String object "Alice"                             │
│ addr 5000: String object "Bob"                               │
│                                                              │
│ addr 3000: [header|length=2] [6000] [7000]                   │
│            (matrix outer array - stores references)          │
│                                                              │
│ addr 6000: [header|length=3] [0] [0] [0]                     │
│            (matrix row 0)                                    │
│                                                              │
│ addr 7000: [header|length=3] [0] [0] [0]                     │
│            (matrix row 1)                                    │
│                                                              │
│            [String Pool]                                     │
│            "Alice", "Bob" might be interned here             │
└──────────────────────────────────────────────────────────────┘
```

Every concept — stack vs heap, reference variables, contiguous storage, array headers, string pool, array-of-arrays — is visible in this single picture. This is exactly what exists in your JVM's memory when that code runs.